

Software Requirements Specification (SRS)

SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 **Standard Followed:** IEEE 830-1998 / ISO/IEC/IEEE 29148:2018 (hybrid structure) **Status:** Implementation-Ready Draft **Based On:** Product Vision v1.0, PRD v1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for **SentinelCut**, an AI-powered platform that detects and censors profanity in audio and video files across English, Hindi, Hinglish, and mixed-language content. This document is written for use by the development team, QA team, and project evaluators as the authoritative technical requirements reference. It is intended to be sufficiently detailed that a developer can begin implementation without requiring further requirement clarification.

1.2 Scope

The system, SentinelCut, shall:

- Accept audio/video uploads from users.
- Transcribe speech using automatic speech recognition (ASR) with multilingual support.
- Detect profane/offensive words using a hybrid dictionary + context-aware NLP approach.
- Allow users to review and edit detections via a timeline interface.
- Generate a censored output file (beep/mute/custom sound) with preserved audio-video synchronization.
- Provide user account management, a dashboard, and a lightweight admin panel.

The system shall **not**, in this version, provide: real-time/live-stream censorship, AI voice cloning, mobile native apps, or third-party platform integrations (Zoom/Discord/OBS/YouTube). These are documented as future scope in the Product Vision document.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
ASR	Automatic Speech Recognition
NLP	Natural Language Processing
FR	Functional Requirement
NFR	Non-Functional Requirement
MVP	Minimum Viable Product
RBAC	Role-Based Access Control
JWT	JSON Web Token
API	Application Programming Interface
Hinglish	Code-mixed Hindi-English spoken/written language
Code-switching	Alternating between two or more languages within a single conversation/sentence
Profanity Detection Engine (PDE)	The subsystem responsible for identifying offensive words/phrases from transcribed text
Timeline Editor	The UI component allowing users to review/edit detected profanity instances
Detection Instance	A single flagged word/phrase with associated timestamp, confidence score, and source
Mux/Muxing	The process of merging processed audio back into a video container
SRS	Software Requirements Specification

1.4 References

- Product Vision Document v1.0 (SentinelCut)
- Product Requirements Document (PRD) v1.0 (SentinelCut)
- IEEE 830-1998, Recommended Practice for Software Requirements Specifications
- ISO/IEC/IEEE 29148:2018, Systems and Software Engineering — Requirements Engineering

1.5 Overview of Document

Section 2 provides an overall description of the product, users, and constraints. Section 3 details system features and functional requirements with unique IDs. Section 4 specifies external interface requirements. Section 5 specifies non-functional requirements (performance, security, usability, etc.). Section 6 covers appendices, including traceability notes.

2. Overall Description

2.1 Product Perspective

SentinelCut is a new, standalone, cloud-hosted SaaS web application. It is not a replacement for professional NLE (non-linear editing) software but a complementary automation layer for a specific task: profanity detection and censorship. The system architecture consists of four logical subsystems:

1. **Frontend Web Application** — user-facing UI (upload, dashboard, timeline editor, admin panel).
2. **Backend API Service** — handles auth, business logic, job orchestration, database access.
3. **AI Processing Pipeline** — ASR, language detection, profanity detection, audio/video processing (runs as background/queued jobs).
4. **Storage & Database Layer** — persistent storage for files, transcripts, user data, and metadata.

2.2 Product Functions (Summary)

- User registration/authentication (including guest mode)
- File upload with format/size validation
- Automatic, Custom, and Combined profanity detection modes
- Multilingual ASR and language auto-detection
- Timeline-based manual review and editing of detections
- Audio/video censorship rendering with sync preservation
- Multi-format export (video, audio, subtitle, transcript, JSON)
- Admin management of users, profanity database, and logs

2.3 User Characteristics

User Class	Technical Proficiency	Frequency of Use	Notes
Guest User	Low-Medium	One-time/occasional	No account; limited feature access; session-based
Registered Creator	Low-Medium	Regular (weekly)	Primary user base; expects simple, fast workflow
Professional Editor	Medium-High	Frequent (daily/weekly)	Values precision, control, and export flexibility
Admin	High	Occasional	Manages system-level data and monitors platform health

2.4 Assumptions and Dependencies

Assumptions

- Users have legal rights to uploaded content.
- Users have a stable internet connection for upload/download.
- A reasonable accuracy bar ($\geq 85\%$ recall) is acceptable for MVP; the timeline editor compensates for residual errors.

Dependencies

- Availability of a suitable multilingual open-source or free-tier ASR engine (finalized in Tech Stack Justification document).
- Availability of FFmpeg for audio/video processing.
- Third-party OAuth provider (Google) uptime for social login.
- Cloud hosting/storage free-tier availability for academic deployment.

2.5 Constraints

- Must be buildable within an academic project timeline using primarily free/open-source tools.
- Must operate acceptably on CPU-only infrastructure, with GPU as an optional performance enhancement, not a hard requirement.
- No large proprietary Hindi/Hinglish profanity-labeled dataset is available; the system relies on curated dictionaries, open datasets, and normalization heuristics.

3. System Features and Functional Requirements

Each feature is documented with a unique ID pattern: **SRS-FR-[Module]-[Number]**.

3.1 Module: Authentication (AUTH)

Description: Enables user registration, login, and session management, including guest access.

Req ID	Requirement	Priority
SRS-FR-AUTH-01	The system shall allow a user to register using email and password.	Must
SRS-FR-AUTH-02	The system shall allow a user to log in using Google OAuth 2.0.	Must
SRS-FR-AUTH-03	The system shall allow password reset via a time-limited emailed link (expires in 30 minutes).	Must
SRS-FR-AUTH-04	The system shall support Guest Mode, issuing a temporary session token without requiring registration.	Must
SRS-FR-AUTH-05	The system shall enforce password complexity (minimum 8 characters, at least 1 number, 1 special character).	Must
SRS-FR-AUTH-06	The system shall issue a JWT access token upon successful authentication, valid for a configurable duration (default 60 minutes), with refresh token support.	Must
SRS-FR-AUTH-07	The system shall expire guest sessions and associated temporary files after a configurable period (default 24 hours).	Should

3.2 Module: File Upload (UPLD)

Req ID	Requirement	Priority
SRS-FR-UPLD-01	The system shall accept video files in formats: MP4, MOV, MKV, AVI.	Must
SRS-FR-UPLD-02	The system shall accept audio files in formats: MP3, WAV, AAC, FLAC, M4A.	Must
SRS-FR-UPLD-03	The system shall support drag-and-drop and manual file browser upload.	Must
SRS-FR-UPLD-04	The system shall reject files exceeding a configurable maximum size (default 500MB) with a descriptive error.	Must
SRS-FR-UPLD-05	The system shall validate file format server-side (not relying solely on file extension) prior to queuing.	Must
SRS-FR-UPLD-06	The system shall display real-time upload progress as a percentage.	Should
SRS-FR-UPLD-07	The system shall scan uploaded files for malware prior to processing.	Should

3.3 Module: Detection Configuration (DTCT-CFG)

Req ID	Requirement	Priority
SRS-FR-DTCT-CFG-01	The system shall provide three detection modes: Automatic, Custom, Combined.	Must
SRS-FR-DTCT-CFG-02	In Custom mode, the system shall accept a user-supplied list of words/phrases (comma or newline separated).	Must
SRS-FR-DTCT-CFG-03	In Combined mode, the system shall merge automatic-database results with custom-list results, removing duplicates.	Must
SRS-FR-DTCT-CFG-04	The system shall allow registered users to save and reuse custom word lists as named presets.	Should
SRS-FR-DTCT-CFG-05	The system shall support Unicode (Devanagari script) input for custom word lists.	Must

3.4 Module: Language Detection & ASR (LANG-ASR)

Req ID	Requirement	Priority
SRS-FR-LANG-01	The system shall automatically detect the primary spoken language(s) in an uploaded file: English, Hindi, Hinglish, or mixed.	Must
SRS-FR-LANG-02	The system shall select an ASR configuration appropriate to the detected language(s).	Must
SRS-FR-LANG-03	The system shall support code-switching detection within a single file/sentence.	Should
SRS-FR-LANG-04	The system shall allow the user to manually override the detected language, triggering re-transcription.	Should
SRS-FR-LANG-05	The system shall generate a time-aligned transcript (word-level or segment-level timestamps) from the ASR output.	Must

3.5 Module: Profanity Detection Engine (PDE)

Req ID	Requirement	Priority
SRS-FR-PDE-01	The system shall detect profane/offensive words using a hybrid dictionary-matching plus context-aware model approach.	Must
SRS-FR-PDE-02	The system shall normalize repeated-character variants (e.g., "fuuuuck" → "fuck") prior to matching.	Must
SRS-FR-PDE-03	The system shall detect common transliterated/abbreviated slang terms via a curated slang dictionary.	Must
SRS-FR-PDE-04	The system shall normalize leetspeak/symbol-substituted text (e.g., "@a" → "a") prior to matching.	Must
SRS-FR-PDE-05	The system shall assign a confidence score (0.0–1.0) to each detection instance.	Must
SRS-FR-PDE-06	The system shall allow filtering of displayed detections by a user-configurable confidence threshold.	Should
SRS-FR-PDE-07	The system shall tag each detection with its source (dictionary, custom-list, or model-inferred).	Should
SRS-FR-PDE-08	The profanity dictionary shall be extensible by administrators without requiring a code deployment.	Should

3.6 Module: Timeline Editor (TLE)

Req ID	Requirement	Priority
SRS-FR-TLE-01	The system shall display the full transcript with detected instances visually highlighted.	Must
SRS-FR-TLE-02	The system shall synchronize transcript display with audio/video playback position.	Must
SRS-FR-TLE-03	The system shall allow the user to delete (dismiss) a detection instance.	Must
SRS-FR-TLE-04	The system shall allow the user to manually add a new detection instance by selecting text or a timestamp range.	Must
SRS-FR-TLE-05	The system shall allow the user to change the replacement mode for an individual detection instance.	Must
SRS-FR-TLE-06	The system shall allow the user to fine-tune the start/end timestamp of a detection instance.	Should
SRS-FR-TLE-07	The system shall support "Regenerate Output" that reprocesses the file based on current edits.	Must
SRS-FR-TLE-08	The system shall auto-save timeline edit drafts periodically (e.g., every 30 seconds) to prevent data loss.	Should
SRS-FR-TLE-09	The system shall support undo/redo for at least the last 10 edit actions.	Could

3.7 Module: Replacement / Censorship (REPL)

Req ID	Requirement	Priority
SRS-FR-REPL-01	The system shall support Beep replacement with configurable tone frequency.	Must
SRS-FR-REPL-02	The system shall support Mute (silence) replacement.	Must
SRS-FR-REPL-03	The system shall support Custom uploaded audio-clip replacement.	Should
SRS-FR-REPL-04	The system shall reserve an extensible interface for future AI voice-word replacement (not implemented in current release).	Could
SRS-FR-REPL-05	The system shall ensure replacement audio duration matches the original detected segment duration (± 50 ms tolerance).	Must

3.8 Module: Export (EXPT)

Req ID	Requirement	Priority
SRS-FR-EXPT-01	The system shall export a censored video file preserving original container format where feasible.	Must
SRS-FR-EXPT-02	The system shall export a censored audio-only file.	Must
SRS-FR-EXPT-03	The system shall export a subtitle file in .srt format reflecting censorship markers.	Should
SRS-FR-EXPT-04	The system shall export a plain-text transcript (.txt).	Must
SRS-FR-EXPT-05	The system shall export a JSON metadata file containing all detection instances, timestamps, confidence scores, and user actions taken.	Must

3.9 Module: Dashboard (DASH)

Req ID	Requirement	Priority
SRS-FR-DASH-01	The system shall display a list of the user's recent files with processing status.	Must
SRS-FR-DASH-02	The system shall provide searchable/filterable file history.	Should
SRS-FR-DASH-03	The system shall display remaining credits/usage quota.	Should
SRS-FR-DASH-04	The system shall display current storage usage against the user's limit.	Should
SRS-FR-DASH-05	The system shall display live status of the processing queue for in-progress jobs.	Must

3.10 Module: Admin Panel (ADM)

Req ID	Requirement	Priority
SRS-FR-ADM-01	The system shall allow administrators to view and manage registered users (view, disable, delete).	Should
SRS-FR-ADM-02	The system shall allow administrators to add, edit, or remove entries in the global profanity dictionary.	Should
SRS-FR-ADM-03	The system shall allow administrators to view uploaded file records and processing logs.	Should
SRS-FR-ADM-04	The system shall present basic analytics: uploads per day, average processing time, detection statistics.	Could
SRS-FR-ADM-05	The system shall log all admin actions with admin identity and timestamp for auditability.	Should
SRS-FR-ADM-06	The system shall allow administrators to view user-submitted feedback.	Could

4. External Interface Requirements

4.1 User Interfaces

- Responsive web application (desktop and tablet breakpoints minimum) built with a component-based framework.

- Key screens: Landing Page, Signup/Login, Dashboard, Upload, Detection/Output Configuration, Processing Status, Timeline Editor, Export/Download, Settings/Profile, Admin Dashboard.
- Support for Light Mode and Dark Mode (Should priority).

4.2 Hardware Interfaces

- No direct hardware interface required. Optional GPU utilization on the backend/AI processing server for accelerated inference, abstracted from the end user.

4.3 Software Interfaces

Interface	Purpose
ASR Engine (final selection documented in Tech Stack Justification)	Speech-to-text transcription
FFmpeg	Audio extraction, audio/video processing, muxing, format conversion
Google OAuth 2.0 API	Social login
Email Service (transactional email provider)	Password reset, notifications
Object Storage Service	Storing uploaded/processed media files
Relational Database	Storing user, file, and detection metadata
Malware Scanning Service/Library	Uploaded file safety validation

4.4 Communication Interfaces

- RESTful API over HTTPS between frontend and backend.
- WebSocket or polling-based mechanism for real-time processing status updates.
- Secure, authenticated signed URLs for file upload/download to/from object storage.

5. Non-Functional Requirements

5.1 Performance Requirements

Req ID	Requirement
SRS-NFR-PERF-01	The system shall process a standard-definition file in no more than 2x its real-time duration under normal load.
SRS-NFR-PERF-02	The dashboard shall load within 2 seconds for a user account with up to 100 historical files.
SRS-NFR-PERF-03	Processing status updates shall propagate to the UI within 5 seconds of a state change.
SRS-NFR-PERF-04	The system shall support at least [N, e.g., 5] concurrent processing jobs via a queue without degrading API responsiveness (target scaled to available academic infrastructure).

5.2 Security Requirements

Req ID	Requirement
SRS-NFR-SEC-01	All client-server communication shall occur over HTTPS/TLS.
SRS-NFR-SEC-02	The system shall use JWT-based authentication with short-lived access tokens and refresh tokens.
SRS-NFR-SEC-03	The system shall validate and sanitize all user inputs (file uploads, custom word lists, form fields) to prevent injection attacks.
SRS-NFR-SEC-04	The system shall enforce Role-Based Access Control (RBAC) distinguishing Guest, Registered User, and Admin roles.
SRS-NFR-SEC-05	The system shall encrypt sensitive data at rest (e.g., password hashes using a strong algorithm such as bcrypt/argon2).
SRS-NFR-SEC-06	The system shall implement rate limiting on authentication and upload endpoints to mitigate abuse.
SRS-NFR-SEC-07	The system shall scan uploaded files for malware prior to processing.
SRS-NFR-SEC-08	The system shall follow OWASP Top 10 mitigation practices for web application security.

5.3 Usability Requirements

Req ID	Requirement
SRS-NFR-USE-01	A first-time user shall be able to complete the upload → detect → export flow without external documentation, in a usability test with representative users.
SRS-NFR-USE-02	The UI shall meet baseline WCAG 2.1 AA contrast and keyboard-navigation standards where feasible.
SRS-NFR-USE-03	Error messages shall be specific and actionable (e.g., naming the exact unsupported format rather than a generic error).

5.4 Reliability & Availability Requirements

Req ID	Requirement
SRS-NFR-REL-01	The system shall target 99% uptime for the web application (academic/demo-scale target).
SRS-NFR-REL-02	Failed processing jobs shall be retried automatically up to a configurable retry count before being marked as failed and surfaced to the user.
SRS-NFR-REL-03	Partial timeline edits shall be preserved (auto-saved) in the event of a client-side crash or disconnect.

5.5 Maintainability Requirements

Req ID	Requirement
SRS-NFR-MAINT-01	The codebase shall follow a modular, layered architecture (Repository Pattern for data access, service layer for business logic) to enable independent updates to the ASR/NLP components.
SRS-NFR-MAINT-02	All modules shall include inline documentation/comments and adhere to a documented coding standard (see Coding Standards document).
SRS-NFR-MAINT-03	The system shall externalize configuration (e.g., max file size, token expiry, confidence thresholds) rather than hardcoding values.

5.6 Portability Requirements

Req ID	Requirement
SRS-NFR-PORT-01	The backend and AI processing services shall be containerized (Docker) to ensure consistent deployment across environments.
SRS-NFR-PORT-02	The system shall avoid platform-specific dependencies that would prevent deployment across common cloud providers or local infrastructure.

5.7 Legal & Compliance Requirements

Req ID	Requirement
SRS-NFR-LEGAL-01	The system shall include a Privacy Policy and Terms & Conditions accepted at signup.
SRS-NFR-LEGAL-02	The system shall provide users the ability to delete their account and associated data.
SRS-NFR-LEGAL-03	The system shall not retain guest-mode files beyond the configured expiry window.

6. Appendices

6.1 Requirement Priority Legend

Priority	Meaning
Must	Required for MVP; system is non-functional without it
Should	Important, targeted for near-term release, not blocking MVP
Could	Desirable enhancement, lowest priority

6.2 Traceability Note

Each functional requirement in Section 3 traces back to a corresponding user story/feature in the PRD (v1.0) and supports one or more goals defined in the Product Vision (v1.0). A full Requirements Traceability Matrix (RTM) mapping Vision → PRD → SRS → Test Cases is provided as a separate deliverable in the documentation package.

6.3 Open Items for Architecture Phase

- Final ASR engine selection and justification (Tech Stack Justification document).
- Detailed database schema (Database Design document).

- API endpoint specification (API Design document).
- Confirmed queueing/background job technology choice (System Architecture document).

6.4 Document Revision History

Version	Date	Change
1.0	Current	Initial implementation-ready SRS derived from Product Vision and PRD

7. Next Document

Document 4: Tech Stack Justification — comparative evaluation and final recommendation for ASR engine, profanity detection architecture, backend framework, frontend framework, database, storage, and deployment platforms, directly informing the System Architecture document that follows.

End of Software Requirements Specification